I²C Y MANEJO DE PANTALLAS ALFANUMÉRICAS EN ARDUINO

Enfoque técnico con módulo LCD 16×2 y biblioteca LiquidCrystal_I2C de Frank de Brabander

Universidad del Magdalena
Programa de referencia: Automatización Industrial
Documento técnico introductorio

Profesor: Gian Carlo Daraviña Peña MS.c

El bus Inter-Integrated Circuit (I²C) constituye un mecanismo de comunicación serial síncrona de dos hilos orientado al intercambio de datos entre circuitos integrados. En plataformas Arduino, la implementación de I²C simplifica la conexión de dispositivos periféricos y reduce el consumo de pines digitales. Dentro de ese conjunto de aplicaciones, las pantallas LCD alfanuméricas con adaptador I²C permiten la visualización local de variables, estados de proceso, alarmas y mensajes de servicio con una interfaz eléctrica compacta y de bajo costo.

La combinación de una pantalla LCD compatible con el controlador HD44780 y un expansor de entrada/salida, generalmente basado en PCF8574, habilita el control del módulo mediante las líneas SDA y SCL. La biblioteca LiquidCrystal_I2C desarrollada por Frank de Brabander abstrae la secuencia de inicialización, direccionamiento y escritura, lo que favorece una estructura de software modular y mantenible.

1. Fundamentos del protocolo I²C

I²C es un protocolo serial síncrono de arquitectura maestro-esclavo y topología multipunto. La transmisión utiliza dos señales: SDA para los datos y SCL para la sincronización temporal. Ambas líneas operan con salida de colector abierto o drenador abierto, razón por la cual requieren resistencias pull-up hacia la tensión de alimentación. En un escenario típico con Arduino UNO y un módulo LCD I²C comercial, dichas resistencias suelen venir integradas en la placa adaptadora.

La comunicación comienza con una condición START generada por el maestro. A continuación, se transmite una dirección de 7 bits junto con un bit de lectura/escritura. Cada byte enviado es seguido por un bit de reconocimiento ACK o NACK. La transacción finaliza con una condición STOP. Las velocidades normalizadas más comunes corresponden a 100 kHz en modo estándar y 400 kHz en modo rápido. Para aplicaciones de visualización, el modo estándar resulta suficiente en la mayoría de las implementaciones.

Las principales ventajas de I²C en instrumentación y automatización incluyen reducción de cableado, expansión sencilla de periféricos, coexistencia de múltiples dispositivos sobre el mismo bus y facilidad de integración con sensores, memorias, convertidores y módulos de interfaz humana. Como limitaciones técnicas, deben considerarse la longitud del bus, la capacitancia total de línea, la correcta selección de resistencias pull-up y la compatibilidad de niveles lógicos entre nodos.

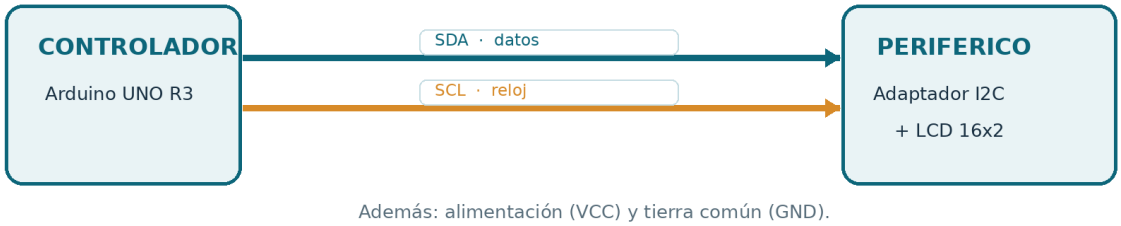


Figura 1. Relación funcional entre Arduino y un periférico LCD mediante las señales SDA y SCL, además de la alimentación VCC y GND.

2. Pantallas LCD alfanuméricas con interfaz I²C

La pantalla LCD alfanumérica 16×2 constituye uno de los módulos más difundidos para despliegue de texto en sistemas embebidos. El elemento frontal incorpora la matriz de cristal líquido y la interfaz paralela basada en el controlador HD44780 o un equivalente funcional. En la parte posterior se integra una tarjeta adaptadora I²C, conocida como backpack, que convierte la interfaz paralela en comunicación serial y expone cuatro terminales: GND, VCC, SDA y SCL.

En la tarjeta adaptadora se localiza con frecuencia un circuito integrado PCF8574 o PCF8574A, un potenciómetro multivuelta o de ajuste fino para el contraste, y un circuito de control de retroiluminación. La dirección I²C depende del encapsulado y del estado de los puentes de configuración de dirección. Entre las direcciones más comunes se encuentran 0x27 y 0x3F. La identificación definitiva debe realizarse mediante un escáner I²C cuando exista incertidumbre sobre la dirección efectiva del módulo.



Figura 2. Vista frontal y posterior de una pantalla LCD 16×2 con adaptador I²C. En la cara posterior se observan los terminales GND, VCC, SDA y SCL, el circuito expansor y el ajuste de contraste.

3. Integración eléctrica con Arduino

En Arduino UNO y Arduino Nano, la línea SDA corresponde al pin analógico A4 y la línea SCL al pin analógico A5. En Arduino Mega, SDA y SCL se ubican en los pines 20 y 21, respectivamente. La pantalla debe alimentarse con VCC y GND compartiendo referencia de tierra con la placa controladora. El conexionado básico queda definido por la correspondencia GND–GND, VCC–5 V, SDA–SDA y SCL–SCL.

La interfaz I²C no requiere la conexión directa de los dieciséis pines del módulo LCD al microcontrolador, lo cual representa una reducción significativa del cableado respecto de la conexión paralela tradicional.

Debe verificarse la tensión de alimentación admitida por el módulo. En la mayoría de las placas comerciales orientadas a Arduino, la operación a 5 V es el caso nominal. Cuando exista coexistencia con dispositivos de 3,3 V, la evaluación de compatibilidad de niveles lógicos resulta obligatoria.

La presencia de caracteres distorsionados, pantalla en blanco o ausencia de detección en el bus suele asociarse con inversión de cables, dirección I²C incorrecta, contraste mal ajustado o defectos de alimentación. También debe contemplarse la posible duplicidad de direcciones en el mismo bus cuando se conectan múltiples dispositivos I²C.

4. Biblioteca LiquidCrystal_I2C de Frank de Brabander

La biblioteca LiquidCrystal_I2C de Frank de Brabander proporciona una interfaz orientada a objetos para el control de pantallas LCD alfanuméricas conectadas por medio de un adaptador I²C. La instancia del objeto requiere la dirección del dispositivo y la geometría de la pantalla. Para un módulo de 16 columnas y 2 filas con dirección 0x27, la declaración habitual adopta la forma LiquidCrystal_I2C lcd(0x27, 16, 2);.

Las funciones más empleadas en un esquema de control o monitoreo se describen a continuación. La función init() inicializa el módulo y prepara el canal de comunicación. La función backlight() habilita la retroiluminación y noBacklight() la deshabilita. La función clear() limpia el contenido de la memoria visible, mientras home() reposiciona el cursor en la esquina superior izquierda. La función setCursor(columna, fila) define la coordenada de escritura. La función print() envía texto o variables convertidas a formato imprimible. Las funciones cursor(), noCursor(), blink() y noBlink() gestionan la visibilidad del cursor y su parpadeo. Las funciones scrollDisplayLeft() y scrollDisplayRight() desplazan el contenido. La función createChar() permite almacenar caracteres personalizados en la memoria CGRAM del controlador. Finalmente, write() posibilita la escritura directa de códigos de carácter, incluidos los personalizados.

Desde la perspectiva de diseño de software, el empleo de funciones encapsuladas para inicialización, limpieza parcial, actualización de variables y presentación de mensajes favorece la reutilización del código y reduce la repetición de instrucciones dentro de loop(). Tal organización resulta especialmente útil en prácticas de laboratorio, adquisición de datos y paneles de supervisión local.

5. Estructura funcional del programa

La estructura recomendada consiste en separar la lógica de visualización en funciones específicas. Una función de inicialización concentra la configuración básica del LCD; una función de bienvenida gestiona la presentación inicial; una función de actualización recibe valores de proceso; y funciones auxiliares

adicionales permiten limpiar líneas, mostrar estados o generar alarmas. Con este enfoque, loop() mantiene un papel acotado y facilita la integración con sensores, temporizadores y rutinas de control.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void lcdInicializar();
void lcdMensajeInicio();
void lcdLimpiarLinea(uint8_t fila);
void lcdMostrarTemperatura(float valor);
void lcdMostrarEstado(bool activo);

void setup() {
  lcdInicializar();
  lcdMensajeInicio();
  delay(1500);
}

void loop() {
  float temperatura = 24.6;
  bool estadoSistema = true;

  lcdMostrarTemperatura(temperatura);
  lcdMostrarEstado(estadoSistema);
  delay(1000);
}

void lcdInicializar() {
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.home();
}

void lcdMensajeInicio() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Interfaz I2C");
  lcd.setCursor(0, 1);
  lcd.print("LCD 16x2 listo");
}

void lcdLimpiarLinea(uint8_t fila) {
  lcd.setCursor(0, fila);
  lcd.print("                ");
}

void lcdMostrarTemperatura(float valor) {
  lcdLimpiarLinea(0);
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(valor, 1);
  lcd.print((char)223);
  lcd.print("C");
```

```
}

void lcdMostrarEstado(bool activo) {
  lcdLimpiarLinea(1);
  lcd.setCursor(0, 1);
  lcd.print("Estado: ");
  if (activo) {
    lcd.print("Activo");
  } else {
    lcd.print("Inactivo");
  }
}
```

Código 1. Ejemplo de organización funcional para inicialización y actualización de una pantalla LCD 16×2 bajo la biblioteca LiquidCrystal_I2C.

La función lcdInicializar() agrupa la secuencia de arranque del dispositivo. La función lcdMensajeInicio() reserva un mensaje de identificación o diagnóstico inicial. La función lcdLimpiarLinea() realiza una limpieza selectiva por fila, evitando el uso reiterado de clear(), cuya ejecución puede producir parpadeo visible en determinadas aplicaciones. La función lcdMostrarTemperatura() actualiza una variable de proceso con una resolución decimal. La función lcdMostrarEstado() traduce un valor lógico a una condición legible en pantalla.

6. Manejo avanzado: caracteres personalizados y actualización eficiente

El controlador HD44780 dispone de una memoria CGRAM para el almacenamiento de hasta ocho caracteres personalizados de 5×8 puntos. Esta característica resulta útil para representar iconos de proceso, flechas, indicadores de nivel o símbolos simplificados. El acceso se realiza mediante createChar(posición, matriz) y la posterior escritura del código con write(posición).

```
byte iconoCheck[8] = {
  B00000,
  B00001,
  B00011,
  B10110,
  B11100,
  B01000,
  B00000,
  B00000
};

void lcdCrearIconos() {
  lcd.createChar(0, iconoCheck);
}

void lcdMostrarConfirmacion() {
  lcd.setCursor(15, 1);
  lcd.write(0);
```

```
}
```

Código 2. Definición de un carácter personalizado y posterior escritura en pantalla.

En aplicaciones con actualización periódica de variables, se recomienda evitar el borrado total del display en cada ciclo. La práctica más eficiente consiste en sobrescribir únicamente las posiciones modificadas o limpiar líneas concretas. Este criterio minimiza el parpadeo y mejora la estabilidad visual del sistema de presentación.

7. Diagnóstico y solución de fallas frecuentes

Ausencia total de texto con retroiluminación activa: posible dirección I²C incorrecta, contraste desajustado o biblioteca no compatible con el mapeo del adaptador. Pantalla completamente llena de bloques oscuros: inicialización no ejecutada correctamente o fallo en la comunicación. Escáner I²C sin detección del dispositivo: error de cableado, falta de alimentación, inversión de SDA y SCL, o módulo defectuoso. Texto truncado o desalineado: geometría configurada incorrectamente, por ejemplo, declaración de 20×4 para un módulo físico de 16×2.

El empleo de un programa escáner I²C previo a la integración definitiva constituye una práctica recomendable. Ese procedimiento permite validar la presencia del dispositivo y obtener la dirección efectiva antes de construir la capa de visualización del proyecto principal.

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Escaneo I2C");
}

void loop() {
  byte error;
  int dispositivos = 0;

  for (byte direccion = 1; direccion < 127; direccion++) {
    Wire.beginTransmission(direccion);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("Dispositivo encontrado en 0x");
      if (direccion < 16) Serial.print("0");
      Serial.println(direccion, HEX);
      dispositivos++;
    }
  }

  if (dispositivos == 0) {
    Serial.println("Sin dispositivos detectados");
```

```
    }

    delay(3000);
  }
```

Código 3. Escáner I²C básico para verificar la dirección del módulo LCD en el bus.

8. Consideraciones de ingeniería y cierre técnico

La implementación de pantallas LCD alfanuméricas con interfaz I²C en Arduino ofrece una solución robusta para visualización local en sistemas embebidos, bancos didácticos y prototipos de automatización. Desde el punto de vista de hardware, la reducción del cableado y la ocupación mínima de pines simplifican el diseño de la interfaz. Desde la perspectiva de software, la biblioteca LiquidCrystal_I2C de Frank de Brabander proporciona una abstracción funcional adecuada para inicialización, posicionamiento del cursor, escritura de datos, control de retroiluminación y creación de caracteres personalizados.

La calidad final de la implementación depende de la correcta identificación de la dirección I²C, del ajuste del contraste, de la estabilidad de la alimentación y de una estructura de programa basada en funciones independientes. Este enfoque favorece el mantenimiento del código, la escalabilidad del proyecto y la integración futura con sensores, actuadores y estrategias de control más complejas.